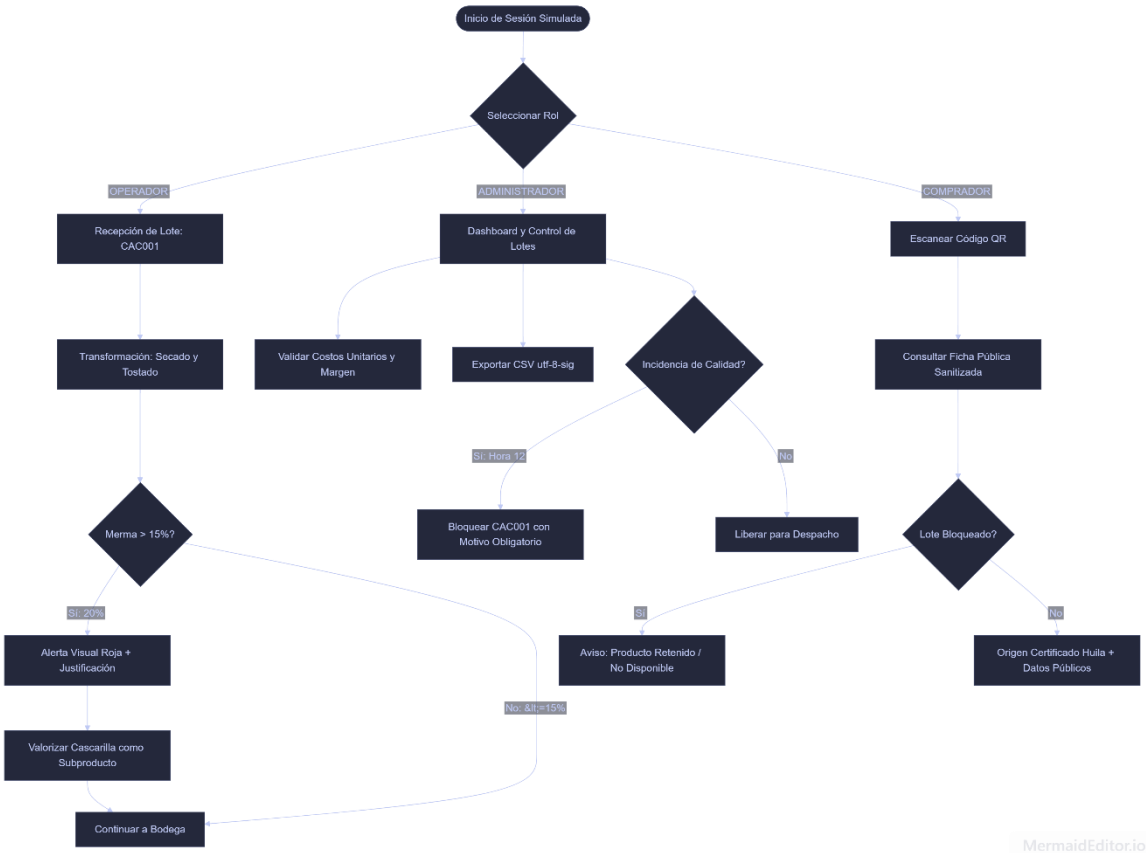
TrazaCao (E03 Diseño del producto y de la solucion)

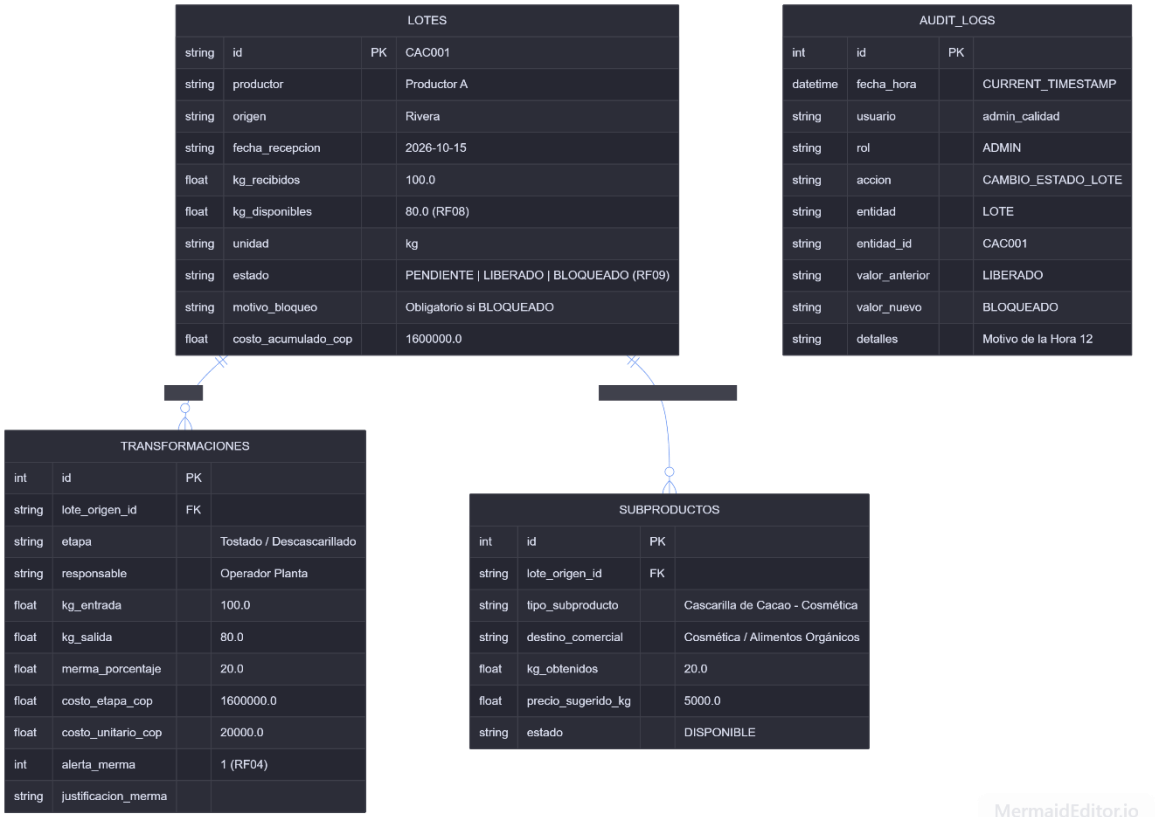
1. Flujo integral del usuario



2. Arquitectura de la solucion



3. Modelo relacional de datos



4. Dos desiciones tecnicas fundamentadas

Decisión 1: Persistencia Embebida en SQLite WAL vs. Motor Cliente-Servidor (PostgreSQL)

- Razón: Garantizar que la aplicación funcione inmediatamente en modo 100% desconectado en cualquier computador portátil o servidor de centro de acopio sin requerir instalación de servicios de red ni contenedores Docker.
- Alternativa Considerada: PostgreSQL en contenedor Docker.
- Limitación Asumida y Mitigación: SQLite bloquea escrituras concurrentes si no está configurado. Se mitigó activando PRAGMA journal_mode = WAL; (lecturas concurrentes no bloquean escrituras y viceversa) y PRAGMA busy_timeout = 5000;.

Decisión 2: Descuento Atómico de Inventario vía SQL Condicional vs. Bloqueo Distribuido

- Razón: El requisito RF08 exige evitar sobreconsumo y reenvíos dobles.
  Un UPDATE ... WHERE kg_disponibles >= consumo garantiza atómicamente a nivel de motor de base de datos que si dos operadores envían la misma petición, solo una descontará el inventario.
- Alternativa Considerada: Bloqueos pesados en memoria con semáforos de Python o SELECT FOR UPDATE.

- Limitación Asumida: Requiere comprobar result.rowcount > 0 en el backend para informar al cliente el rechazo por falta de stock.